Lab-5

Task 1a:

This task is about topological sort using dfs approach. At first made adjacent list and checked if there is a cycle or not. Cycle means it can not be topological sort. For topological sort used visited, path and stack. Where at first called element to 1 then check if it's adjacents are visited. If not the dfs again. After getting our stack simply reversed the values.

Task 1 b: graph = {3:[1], 1:[2], 2:[], 4:[5], 5:[]}

indegree = [0, 1, 1, 0, 0, 1] → for (1, 2, 5)

3 1 2 4 5

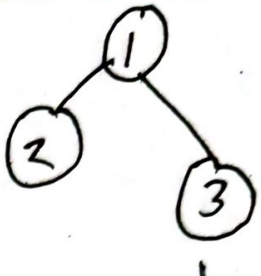
2 5 1 4 3

The graph represent the adjacency matrix and indegree represent degree of each vertex. The topological sort method performs bfs. It dequeues the item and adds vertices with indegree 0 to the queue. After sorting it checks if all vertices are included in the result.

3 1 2 4 5
1 1 2 1 1
0 1 1 0 0

Task-2

graph = {3: [1], 1: [2], 4: [5]}



indegree = {1: [1], 2: [1], 5: [1]}

result = [3, 1, 2, 4, 5]

heap

Here we used heapq. The ~~de~~ defaultdict graph is a defaultdict list that represents the adjacency list and defaultdict of integers to store the indegree of each vertex. The method topological sort performs using priority queue. It initializes an empty heap that iterate all vertices and adds the vertices that have indegree 0. After adding all elements if it's that all are added if not it's a cycle.

Task-3

To do this code we have to use 'Kosaraju' algorithm. At first we initialize a class tracker that keeps track of the start and end time. We first use dfs to calculate finishing time of all vertices. It initializes color of all vertices to white and resets time counter then if it's visited it becomes gray and time increments. The method transpose graph just transpose the edges (u, v) to (v, u). The second dfs pass method perform dfs on transposed graph using dfs to find SCCs. It sets the flag false and resets the time counter. Then it iterates over the vertices in topological order. After that it marks the vertex u as visited as it is explore adjacent vertices.